

Day 4 Lab: Build with Transformers

Applied Deep Learning · YDL 2026

Today we have transformers: models that turn text (and images) into context-aware embeddings, that we can fine-tune to a task, and that we can query in plain language. Let's put all three of those affordances to work. Pick a track: probe and compare these models to understand what they do, ground a model in your own knowledge with retrieval and tools, or build an intriguing app on top of their embeddings or their chat.

The one-sentence brief

Pick a pretrained transformer, then choose a track: Track 1 opens the model up and pits prompting against fine-tuning; Track 2 wires a model to your own documents (retrieval) or to tools (a small agent); Track 3 builds a creative project on the model's embeddings or its chat. Along the way you will touch all three affordances: embeddings, fine-tuning, and querying.

Choose your track

Pick the one that fits what you want to make. All three use the same transformer toolbox and the same small, Colab-friendly models; they just point the work in a different direction.

Track 1: Probe & Compare	Track 2: RAG & Agents	Track 3: Build an App
Interrogate the model itself. Visualize what attention looks at, probe where a concept lives in embedding space, and put approaches head to head: prompt a big model versus fine-tune a small BERT versus a LoRA adapter, on the same task. Your deliverable is a finding about the model, for example an accuracy-versus-cost chart or a map of what a head attends to.	Give a model knowledge or tools. Build a retrieval-augmented question answerer over your own documents (embed, retrieve the relevant chunks, then let the model answer), or a small tool-using agent that can search your files and call a function. Your deliverable is a working RAG assistant or a simple agent.	Build something intriguing with the model's affordances, be it chat or embeddings. A semantic search or recommender, a describe-to-search photo app with CLIP, a styled chat assistant, a fine-tuned tagger, or an explorable map of a collection you love. Same spirit as an open-ended creative project.

The three affordances (you will use all three)

Whatever track you pick, the toolbox lets you do all three of these. Each track just leans on them differently.

Affordance	What it means	Where each track uses it
Embeddings	Turn text or images into vectors you can search, cluster, or classify.	T1: probe embedding geometry. T2: retrieve relevant chunks. T3: search and recommend.
Fine-tuning	Adapt a pretrained model to your task with a small head or a LoRA adapter.	T1: fine-tune BERT and LoRA to compare. T2: (optional) a reranker. T3: specialize the app.

Affordance	What it means	Where each track uses it
Querying	Prompt a large model in natural language; zero-shot and few-shot (in-context).	T1: the prompting baseline. T2: the answer or the agent's reasoning. T3: chat features.

Main tasks

You do not need to turn in or work in the transformer toolbox notebook (*lab_transformer_toolbox.ipynb*), but it has runnable, Colab-ready examples for every tool below, sized for a free T4 GPU.

- Use a pretrained transformer on data you gathered or chose yourself.
- Show it works end to end: turn data into vectors, fine-tune or query a model, and produce a result.
- Produce one artifact that tells a story: a chart, an interactive demo, a working assistant, or a short finding.
- Packaging it as a runnable app is an option, not a requirement, on any track.

Your toolbox (in the notebook)

- **Text to vectors:** a small sentence-embedding model (MiniLM) for search, clustering, and a UMAP map.
- **Look inside attention:** load a small BERT with attention outputs and plot token-to-token heatmaps.
- **Images and text together:** CLIP for image embeddings and zero-shot classification.
- **Fine-tune a small BERT:** train a classification head on a small dataset, then a LoRA adapter, and compare.
- **Query a model:** a keyless local instruct model for chat and few-shot prompting, with an optional API path.
- **Retrieve and reason:** chunk and embed your documents, retrieve top matches, and a minimal tool-using loop.
- **Explore and export:** nearest neighbors, clustering, a UMAP map, and saving vectors or a small app to use locally.

Everything is sized for Colab's free T4 GPU: small models, subsampled data, quantization where it helps, and a tqdm progress bar on every loop. Use Claude freely for visualization, aesthetics, and the optional app.

Project ideas (mix and match)

Each idea names the affordances it combines. **These are just suggestions, feel free to do your own thing and your own idea, so long as it uses a transformer from this week.** Treat them as starting points and bend them toward whatever you find fun.

Track 1: Probe & Compare

Idea	What it is	Combines
Accuracy-versus-cost frontier	Prompt a large model (zero and few-shot), fine-tune a small BERT, and train a LoRA adapter on the same task, then chart accuracy against cost and effort.	querying + fine-tuning

Idea	What it is	Combines
What is it attending to	Map attention heads on ambiguous sentences (like the 'it' example) and describe what each head seems to specialize in.	embeddings / attention
Find a feature	Probe hidden states for a concept (sentiment, tense, topic) and show a direction in embedding space that predicts it.	embeddings
How few examples	Sweep the number of in-context examples (0, 1, 2, 4, 8) and plot how few-shot accuracy climbs.	querying
LoRA rank study	Sweep the LoRA rank r and plot accuracy against the number of trainable parameters, making the low-rank idea concrete.	fine-tuning

Track 2: RAG & Agents

Idea	What it is	Combines
Chat with your documents	Chunk and embed a set of notes, papers, or a PDF, retrieve the most relevant chunks for a question, and let a small model answer with citations.	embeddings + querying
RAG versus no-RAG	Ask the same questions with and without retrieval and check whether grounding reduces made-up answers.	embeddings + querying
A tiny tool-using agent	Give a small model a couple of tools (a calculator and a search-your-files function) in a simple reason-and-act loop.	querying
Structured extraction	Prompt the model to read your documents and fill a schema (fields, tags, or a table) you define.	querying
Rerank the retrieval	Fine-tune or LoRA a small model to re-rank retrieved chunks so the best evidence rises to the top.	fine-tuning + embeddings

Track 3: Build an App

Idea	What it is	Combines
Semantic search engine	Embed your own collection (notes, recipes, arXiv abstracts) and build a similarity search or an explorable 2D map.	embeddings + explore
Describe-to-search photos	Use CLIP to search a set of images by describing them in words, with zero-shot labels.	embeddings (CLIP)
A styled chat assistant	Wrap the local instruct model in a persona or a few-shot format to build a small chat tool.	querying
Tagger as a service	Fine-tune a small classifier head and expose it as a tool that tags incoming text (tickets, emails, posts).	fine-tuning
Embed-anything recommender	Pick a domain, embed its items, and build a nearest-neighbor recommender or an explorable map.	embeddings + explore

Models to use (small and Colab-friendly)

All of these run on a free T4. Prefer the small option first; scale up only if you have headroom. Model names can change, so the toolbox pins working versions.

Role	Suggested model	Why
Text embeddings	sentence-transformers all-MiniLM-L6-v2 (~22M)	Fast, tiny, runs on CPU
Attention / fine-tune	distilbert-base-uncased (~66M)	Small, quick to fine-tune
Image + text	CLIP ViT-B/32 (~150M)	Zero-shot image search
Chat / query (local)	a small instruct model (~0.5 to 3B), 4-bit if needed	Keyless, fits a T4
Chat / query (optional)	a hosted API (OpenAI, Anthropic, or HF Inference)	Faster and stronger, needs a key
Retrieval store	FAISS (faiss-cpu) or Chroma	In-memory vector search

Querying works without any API key: the toolbox ships a small local instruct model. If you add a key (via a Colab secret), the same call transparently upgrades to a hosted model. No student is blocked without one.

More to explore: speech and video

Optional add-ons if your project needs sound or video. Speech models run comfortably on a T4; video generation is compute-heavy, so prefer frame-based tricks or treat the generators as reference (or use a hosted API).

Task	Models to try	Notes
Speech to text	Whisper tiny (~39M), base (~74M), small (~244M); Distil-Whisper	tiny and base run even on CPU
Speech representations	wav2vec 2.0, HuBERT (Meta)	encode raw audio into vectors
Speech translation	SeamlessM4T (Meta)	heavier; use a small variant
Video understanding	CLIP on sampled frames, or VideoMAE	frame-based is cheapest on a T4
Video generation	CogVideoX-2B, LTX-Video, Mochi (open); Sora (API)	compute-heavy; reference, Colab Pro, or API

Where to find data

A few reliable starting points. Skim a source's terms before redistributing, and prefer openly licensed data.

Source	What / how to access	Notes
Hugging Face datasets	One line: sst2, rotten_tomatoes, emotion, ag_news, and thousands more	Great for fine-tuning and probing

Source	What / how to access	Notes
Your own text or PDFs	Notes, papers, a manual, a chat export	Ideal for a RAG assistant
Project Gutenberg	Public-domain books via the Gutendex API (gutendex.com)	Corpora for search or chat
Images	CIFAR and others streamed from Hugging Face, or your own photos	For CLIP search

Audio, image-text pairs, and video

Type	Source	Notes
Audio / speech	Common Voice (Mozilla), LibriSpeech, or HF audio datasets	Many languages; stream from Hugging Face
Music / sound	GTZAN, FSD50K, or your own clips	For audio embeddings or tagging
Image-text pairs	Flickr30k, or a small COCO-captions / Conceptual Captions subset (HF)	For CLIP-style work
Your own images + captions	A folder of images with one caption per file	The simplest image-text set
Video	Your own short clips (sample frames), or a small UCF101 subset	Sample frames, then use the image tools

Exporting from Colab

Your models and vectors live on the Colab machine. To keep or build on them locally, save and download. The toolbox shows both routes:

- **Quick download:** from `google.colab import files`;
`files.download('embeddings.npy')` sends a file to your Downloads.
- **Larger files:** mount Google Drive with `from google.colab import drive`;
`drive.mount('/content/drive')`, then copy files into `/content/drive/MyDrive/`.
- **Optional app:** the toolbox writes a small `app.py`; download it next to your data, then run `streamlit run app.py` locally. Building an app is optional on any track.

Tips

Start small: get one thing working on a slice of data, then scale up. Turn on the T4 GPU (Runtime, Change runtime type). On Track 1 decide up front what comparison would be convincing; on Track 2 get a single grounded answer working before you polish; on Track 3 get one thing explorable early. Lean on the toolbox examples, and use Claude for visualization, aesthetics, and the optional app.

Toolbox libraries: `transformers`, `datasets`, `peft`, `sentence-transformers`, `faiss-cpu` (or `chromadb`), `scikit-learn`, `umap-learn`, `tqdm`, `bitsandbytes` (for 4-bit), and optionally `Streamlit` and a hosted-API SDK.